# Database Testing

# Agenda

1. SQL Introduction
2. Type of DB Management studio
3. SQL Command type
4. Data Types inSQL
5. SQL Queries

# Introduction of SQL

- SQL- Structured Query Language
- SQL is a standard language for storing, manipulating and retrieving data in databases
- SQL keywords are NOT case sensitive Ex. SELECT as select
- Semicolon is the standard way to separate each SQL statement in database systems ex. Select * from employee;
- SQL became a standard of the American National Standards Institute (ANSI) in 1986, and of the International Organization for Standardization (ISO) in 1987

# Types of DB Management studio

# SQL Command type

1. **DDL(Data Definition Language):**

   Allows to work with the Structure or Definitionof the database or tables
   •SQL commands are as follows in DDL: CREATE, ALTER, DROP, TRUNCATE

2. **DML(Data Manipulation Language):**

   To deal with the data itself directly
   •SQL Commands come under DML are as follows: INSERT, UPDATE, DELETE

3. **DQL(Data Query Language):**

   It is used to retrieve data from database
   •SQL Commands that come under DQL are as follows: SELECT

4. **DCL(Data Control Language):**

   Deal with the rights, permissions and other controlsof the database system
   •SQL Commands come under DCL are as follows: GRANT and REVOKE

# Data Types in SQL

## 1 Characters /String

- CHAR(20) -- fixed length
- VARCHAR(255) -- fixed length

## 2 Number

- INT
- REAL FLOT

# SQL Queries

## 1 SQL DDL command-

- **CREATE**

  **Syntax** - CREATE TABLE table_name (column1 datatype, column2 datatype, column3 datatype, .... );

  **Ex**. CREATE TABLE Oneclick
  ( PersonID int,
  LastName varchar(255),
  FirstName varchar(255),
  Address varchar(255) );

- **Drop–** The DROP DATABASE, DROP TABLE statement is used to drop anexisting SQL database, existing table in a database with structure of table of database.

  **Syntax** - DROP TABLE table_name;

  **EX**. DROP TABLE Oneclick;

# SQL Queries

## 1 SQL DDL command-

- **TRUNCATE–** The SQL TRUNCATE TABLE command is used to delete complete data from an existing table but the structure of the table remaining You can also use the DROP TABLE command to delete the complete table but it would remove the complete table structure from the database

**Syntax** – TRUNCATE TABLE table_name;

**EX**. TRUNCATE TABLE Oneclick

- **ALTER –** The ALTER TABLE statement is used to add, drop, or Modifycolumnsin an existing

**Syntax** – ALTER TABLE table_name ADD column_name datatype;

**EX**. ALTER TABLE Oneclick ADD Email varchar(20);

# SQL Queries

## 2 SQL DML command-

- **INSERT –** INSERT statement is used to insert a single record or multiple records into a table in SQ

  **Syntax** – INSERT INTO TN. (CN1, CN2, ...) VALUES (value1, value2, ...);

  **EX**. INSERT INTO Oneclick (FN , LN, Mobile ) Values ('Sumit', 'Narale', 7083208438)

- **SELECT–** Select statement is used to fetch the data from a database table.

  **Syntax** – SELECT * FROM TN

  SELECT CNS. FROM TN.;

  **EX**. SELECT * FROM Oneclick;

  SELECT Fname, Lname FROM Oneclick;

# SQL Queries

## 2 SQL DML command-

- **Aggregate Function – Aggregate used with select statements, they will return some numeric values**

1. **COUNT()** function returns the number of rows that matches.
2. **AVG()** function returns the average value of a numeric column.
3. **SUM()** function returns the total sum of a numeric column.
4. **MAX()** function returns the max number of data from coloumn
5. **MIN()** function returns the min value of a numeric column.

**Syntax –** SELECT COUNT (C.N.) FROM T.N.
SELECT AVG (C.N.) FROM T.N.
SELECT SUM (C.N.) FROM T.N
SELECT MAX (C.N.) FROMT.N.
SELECT MIN (C.N.) FROM T.N.

**Ex.** SELECT COUNT (Marks) FROM Oneclick;
SELECT AVG (Marks) FROM Oneclick;
SELECT SUM (Marks) FROM Oneclick;

# SQL Queries

2 **SQL DML command-**

- **WHERE -** clause is used to specify a condition while fetching the data from a single table

**Syntax –** SELECT * FROMT.N.
WHERE Condition

**Ex.** SELECT * FROMOneclick
WHERE FName = ‘Sumit’;

# SQL Queries

## 2 SQL DML command-

- **AND, OR, NOT –** operators are used to combine multiple conditions to narrow data in an SQL statemen**t.**

**Syntax** - SELECT * FROM T.N.
WHERE Condition AND Condition

**Syntax**- SELECT * FROM T.N.
WHERE Condition OR Condition

**Syntax**- SELECT * FROM T.N.
WHERE NOT Condition

**Ex.** SELECT * FROM Oneclick
Where Fname = 'Sumit' AND Lname = 'Narale' ;

**Ex.** SELECT * FROM Oneclick
Where Fname = 'Sumit' OR Lname = 'Narale' ;

**Ex.** SELECT * FROM Oneclick
Where NOT Fname = 'Sumit' ;

# SQL Queries

## 2 SQL DML command-

- **LIKE –** clause is used to find a value to similar values or pattern using **wildcard operators**. There are two **wildcards** used in conjunction with the **LIKE** operator.

- **Wildcard operates** - 1.**percent sign (%)** à Matches one or more characters.
  2.**underscore ( _ )** à Matches onecharacter.
  3.**[charlist]%** à Matches more characters incharlist
  4.**[ ! charlist]%** à Not matches characters sequence

**Syntax**– SELECT C.N.S FROM T.N.
WHERE C.N. LIKE 'Pattern'

Ex. SELECT * FROM Oneclick
where Fname LIKE 'A%';
Ex. SELECT * FROM Oneclick
where Fname LIKE 'Adi_y';
Ex. SELECT * FROMOneclick
where Fname LIKE '[ AN]% ';

A% = AXXXXXX,
%A = XXXXXXA
%A% = XXXAXXX
AM_N = AMXN
_ATIL = XATIL
[AN]% = AXXXX ,NXXXX

# SQL Queries

2 SQL DML command-

- **BETWEEN** – keyword / operator used to selects values within a given range with a WHERE clause

  **Syntax-** SELECT C.N.S FROMT.N.
  WHERE C.N. BETWEEN Value1 AND Value2

  **Ex.** SELECT * FROM Oneclick
  WHERE Marks BETWEEN 6 AND 9; RESULT- 6,7,8,9

- **IN** keyword / operator allows you to specify multiple values in a WHERE clause

  **Syntax-** SELECT C.N.S FROM T.N.
  WHERE C.N. IN (Value1, Value2, Value3)

  **Ex.** SELECT * FROMOneclick
  WHERE Marks IN ( 6 , 9); RESULT- 6, 9

# SQL Queries

## 2 SQL DML command-

- **IS NULL & IS NOT NULL -** keyword / operator used with WHERE clause.
  To test NULL values, We will have to use the IS NULL and IS NOT NULL operators

**Syntax-** SELECT C.N.S FROM T.N.
WHERE C.N. IS NULL

**Ex**. SELECT * FROM Oneclick
WHERE Marks IS NULL ;

FNAME= AMAN, Sumit, NULL, Harsh, Yagnesh

# SQL Queries

## 2 SQL DML command-

- **Order By-** ORDER BY clause is used to sort the records in your result set

**Syntax -** SELECT C.N.S FROM T.N.
ORDER BY C.N. ASC | DESC;

**Ex.** SELECT * FROM Oneclick
ORDER BY Fname ASC;

- **Alise -** COLUMN ALIASES are used to make changes column headings in your result set easier to read & same as TABLE ALIASES are used to shorten your SQL to make it easier to read.

**Syntax -** C.N. [ AS ] alias_name
T. N. [ AS ]alias_name

**Ex.** SELECT *FROM Oneclick [AS] OneClickItSolution
**Ex.** SELECTFanme as FN FROM Oneclick WHERE FN =“Sumit”

# SQL Queries

## 2 SQL DML command-

- **Update-** UPDATE statement is used to update existing records in a table in a SQL Server database.

**Syntax-** UPDATE T.N.
SET C.N.1 = Value1, C.N. 2=Value2, ...
WHERE condition;

**Ex.** UPDATE Oneclick
SET Fname = ‘Sumit’, Lname= ‘ Patil’
WHERE Fname = ‘ Yagnesh’

- **Delete- DELETE statement is used to delete a single record or multiple records from a table in SQL Server.**

**Syntax-** DELETE FROM T.N. WHERE condition;

**Ex.** DELETE FROM OneclickWHERE Fname = ‘Sumit’

# SQL Queries

## 2 SQL DML command-

- **Union -** UNION operator is used to combine the result sets of 2 or more SELECT statements. It removes duplicate rows between the various SELECT statements. Each SELECT statement within the UNION operator must have the same number of columns in the result sets with similar data types.

**Syntax-**

```
SELECT C.N.S FROM T.N 1.
UNION
SELECT C.N.S FROM T.N 2;
```

**EX.**

```
SELECT * FROM Oneclick.
 UNION
 SELECT * FROM OneclickSolution;
```

# SQL Queries

## 2 SQL DML command-

- **SELECT INTO-** SELECT INTO statement is used to create a table from an existing table by copying the existing table's columns. Newly create table which is temporary.

**Syntax-** SELECT C.N.S
INTO new T. N. FROM old T. N. ;

**EX.** SELECT * INTO
Oneclick FROM OneclickItSolution;

# SQL Table Constrains

SQL constraints are used to **specify rules for data in a table**. Constraints can be used when the table is created with the CREATE TABLE statement

- **NOT NULL -** The NOT NULL constraint forces a column to **NOT accept NULL values**. Because while creating a table if value not present it take NULL value in C.N.

- **UNIQUE -** The UNIQUE constraint ensures, **all values** in a column are **different.**

- **PRIMARY KEY -** The PRIMARY KEY constraint uniquely identifies each record in a table. Primary keys must contain **UNIQUE values**, and **cannot contain NULL** values.

- **FOREIGN KEY -** A FOREIGN KEY is a key used to **link two tables together**. A FOREIGN KEY is a field (or collection of fields) in one table that refers to the PRIMARY KEY in another table.

- **CHECK -** The CHECK constraint is used to **limit the value range** that can be placed in a column. If you define a CHECK constraint on a single column it allows only certain values for this column.

- **DEFAULT -** The DEFAULT constraint is used to provide a **default value** for a column.

# SQL Table Constrains

SQL constraints are used to **specify rules for data in a table**. Constraints can be used when the table is created with the CREATE TABLE statement

**Syntax-** Create table Oneclick (
C.N.1 datatype constraint
C.N*2 datatype constraint*
C.N.*3 datatype constraint, ..)*

**Ex.** CREATE TABLE Oneclick(

```
StudID       int PrimaryKey,
Fname        varchar(255) Not Null,
Lname        varchar(255),
Mockresult   int Check (Mockresult < 20) ,
Location     varchar(255)        Default  'Ahmedabad'
Mobileno     int Unique );
```

# Constrains Difference-

| Primary Key | Foreign Key | Unique Key |
|---|---|---|
| The PRIMARY KEY constraint uniquely identifies each record in a table. Primary keys must contain UNIQUE values | A FOREIGN KEY is a key used to link two tables together. A FOREIGN KEY is a field (or collection of fields) in one table that refers to the PRIMARY KEY in another table. | The UNIQUE constraint ensures that all values in a column are different. |
| Primary key cannot have a NULL value. | Foreign key can accept multiple null value. | Foreign key can accept multiple null value. |
| Each table can have only one primary key. | We can have more than oneforeign key in a table. | Each table can have more than one Unique Constraint |
| Primary key is clustered index | Foreign keys do not automatically create an index, clustered or non-clustered | Unique key is a unique non- clustered index |

# SQL Join

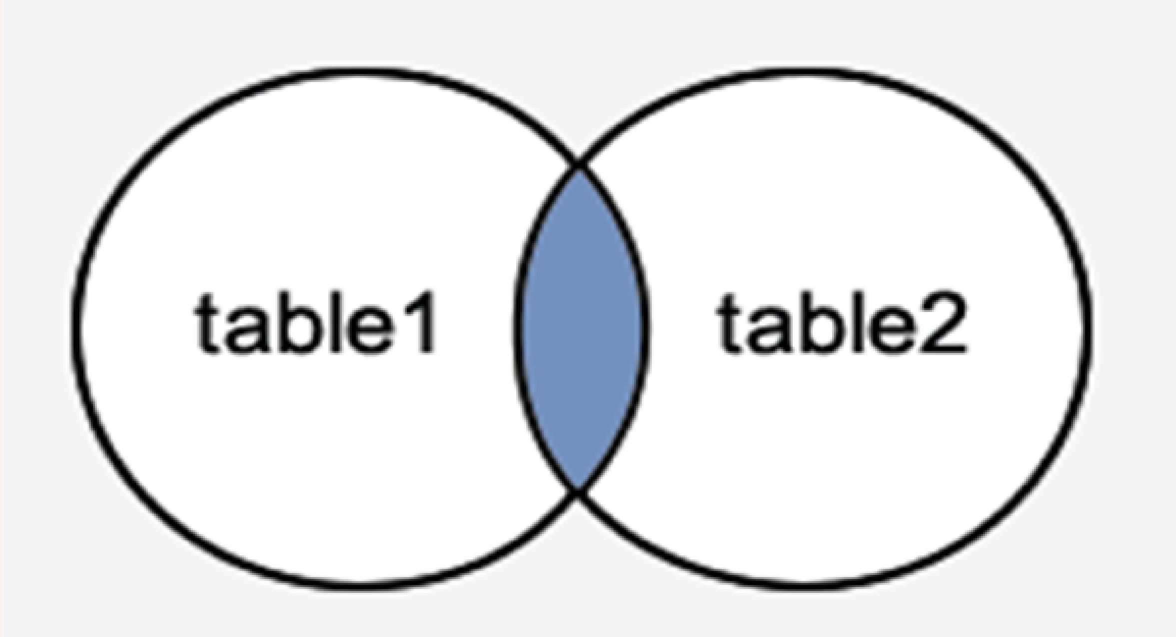
JOINS are used to **retrieve data from multiple tables**. A SQL Server JOIN isperformed whenever two or more tables are joined in a SQLstatement.

There are 5 different types of SQL Server joins:

**1. INNER JOIN** **2. LEFTJOIN** **3. RIGHT JOIN** **4. FULL JOIN** **5. SELF JOIN**

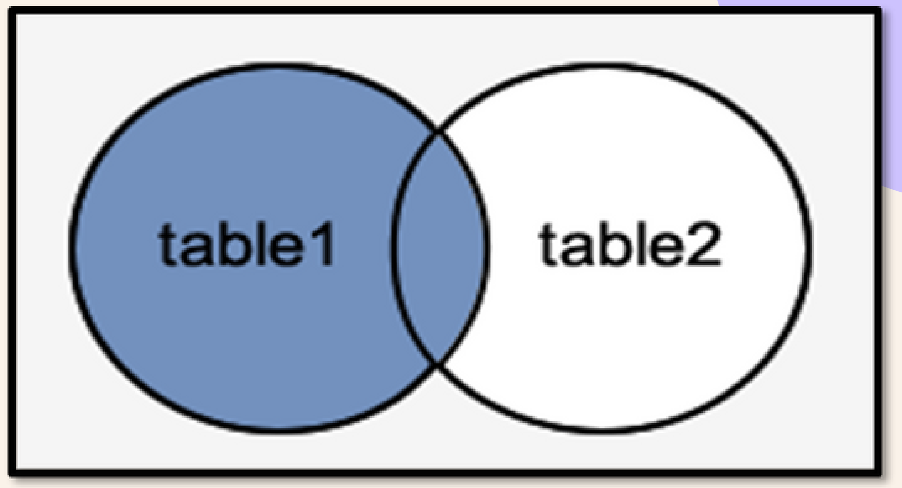
**Syntax :** SELECT TN1 . CN, TN1 . CN, TN2 . CN, TN2 . CN FROM TN1
I INNER JOIN TN2
ON TN1 . CN (Primary Key) = TN2 . CN (ForeignKey)

**INNER JOIN** - It is the most commontype of join. SQL Server INNER JOINS return all rows from multiple tableswhere two table contains common values in the join condition is met.

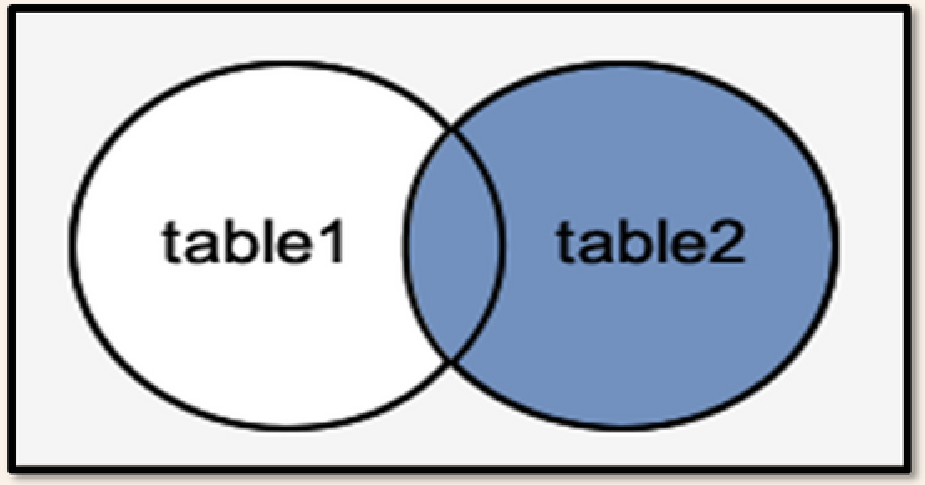


# SQL Join

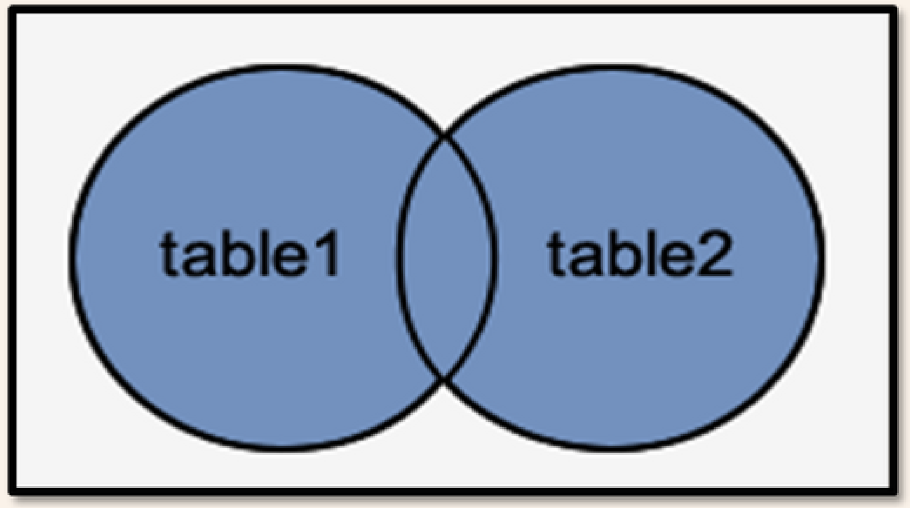
**1.LEFT JOIN -** join returns all rows from the LEFT-hand
table specified in the ON condition
and only those rows from theother table where the
joined fields are equal (join condition is met).




**2.RIGHT JOIN** - This type of join returns all rows from
the RIGHT-hand table specified in the ON condition
and only those rows from the other table where the
joined fields are equal (join condition is met).




**3.FULL JOIN** - This type of joinreturns all rows from the
LEFT- hand table and RIGHT-hand tablewith nulls in place
where the join condition is not met.



# SQL Join

**4. SELF JOIN -** A self join is a join in which a table is joined with itself (which is also called Unarry relationships). The self join can be viewed as a join of two copies of same table. The table is not actually copies but SQL perform the command as though it were

**Syntax** - SELECT TN1 . CN , TN2 . CNFROM TN1 , TN2
WHERE TN1 . CN = TN2 . CN

# SQL Group By

**Group By** - The GROUP BY statement is often used with **aggregate functions** (COUNT, MAX, MIN, SUM, AVG) to group the result set by one or more columns.

**Syntax**- SELECT C.N.S, Aggregate_function (CN) FROM T.N
GROUP BY CN

**Having Clause**- HAVING clause is used in combination with the GROUP BY clause to provide a specific condition.

**Ex.** - SELECT marks, Count (Marks) As result
FROM Oneclick
GROUP BY Marks
HAVING result >1

# Thank you!